Assignment 1: Project Proposal

Overview

This is the project proposal for Group 2 that articulates our agent project vision. This document will serve as a project roadmap for the semester describing the benchmark design (***RescueBench***) and planned agent approach (***Dynamic Emergency Response Allocation Agent***).

Part I: Introduction

1. Problem and Motivation

We address the inability of Large Language Models (LLMs) to reliably solve the Vehicle Routing Problem with Time Windows (VRPTW) during dynamic crises. Disaster response requires parsing unstructured text alongside zero-error mathematical optimization. While LLMs excel semantically, they fail at strict spatial reasoning and multi-step constraint satisfaction [1]. End-to-end LLM solvers hallucinate mathematics, lose track of physical capacities, and output infeasible routes [2, 3]. Recent Operations Research proves LLMs must act as heuristic orchestrators rather than direct mathematical solvers [4], motivating our hybrid architecture.

2. Your Approach

We propose the **Dynamic Emergency Response Allocation Agent**, evaluated on our ***RescueBench*** environment. Our approach uniquely decouples semantic reasoning from mathematical execution. The LLM handles subjective triage, while a deterministic Python validation node calculates travel times and enforces physical constraints before any action is executed. If the LLM proposes an invalid move, the validation node rejects it and returns an explicit error prompt, forcing a self-correcting repair loop.

3. Evaluation Overview

We will deploy an automated Python pipeline testing the agent across 20 JSON-based *RescueBench* scenarios, injecting dynamic text updates to evaluate adaptability. We will measure the Priority-Weighted Resolution Score, Response Efficiency, and Hallucination/Violation Penalties. The agent will be compared against a Greedy Heuristic Solver, a Standard Zero-Shot LLM, and an Ablated Agent stripped of dynamic replanning capabilities.

Part II: Benchmark Design

1. Task Domain

RescueBench operates in Humanitarian Logistics and Emergency Resource Allocation. This domain forces a continuous transition between unstructured semantic reasoning (parsing emergency feeds) and rigorous spatial optimization (updating a mental graph and solving the VRPTW).

2. Task Specifications

We will construct 20 modular JSON environments scaling across 4 difficulty tiers: Basic Triage, Constraint Satisfaction, Ethical Prioritization, and Dynamic Replanning. Here are some example tasks:-

- ➔ **The Multi-Step Logic Trap (Level 4):** A hostile crowd blocks an overpass leading to a fire. The agent must sequence actions: dispatching a Riot Squad to clear the edge constraint *before* routing the fire apparatus.
- ➔ **The Cascading Constraint (Level 2):** Flash flooding washes out a primary bridge while a hospital hits maximum capacity. The agent must recalculate ETAs using perimeter routes and redirect ambulances before deadlines expire.
- ➔ **Severe Scarcity (Level 3):** A mass casualty transit derailment occurs simultaneously with a localized fire. The agent must intentionally delay the lower-priority incident to mathematically maximize global survival weight.

3. Success Criteria

We use three strictly programmable formulas to automatically grade the JSON tool calls:

- **Priority-Weighted Resolution Score (Success Metric):** Maximizes the severity-weighted completion of incidents.

  $$S = \sum_{i=1}^{n} (w_i \times I_i)$$

  (Where w_i is the incident's severity weight [1-10] and I_i is a binary indicator of successful resolution before the deadline).

- **Response Efficiency (Speed Metric):** Evaluates the agent's spatial routing optimization.

  $$E = \frac{T_{opt}}{T_{agent}}$$

  (Where T_{opt} is the theoretically shortest deterministic path time calculated by Dijkstra's algorithm, and T_{agent} is the agent's chosen route time. Score <= 1.0).

- **Hallucination/Violation Penalty (Failure Metric):** A hard count of physically impossible actions proposed by the LLM.

  $$V = \sum c_{invalid}$$

  (Where $c_{invalid}$ counts instances of exceeding vehicle capacities, dispatching units without required capabilities, or traversing blocked edges).

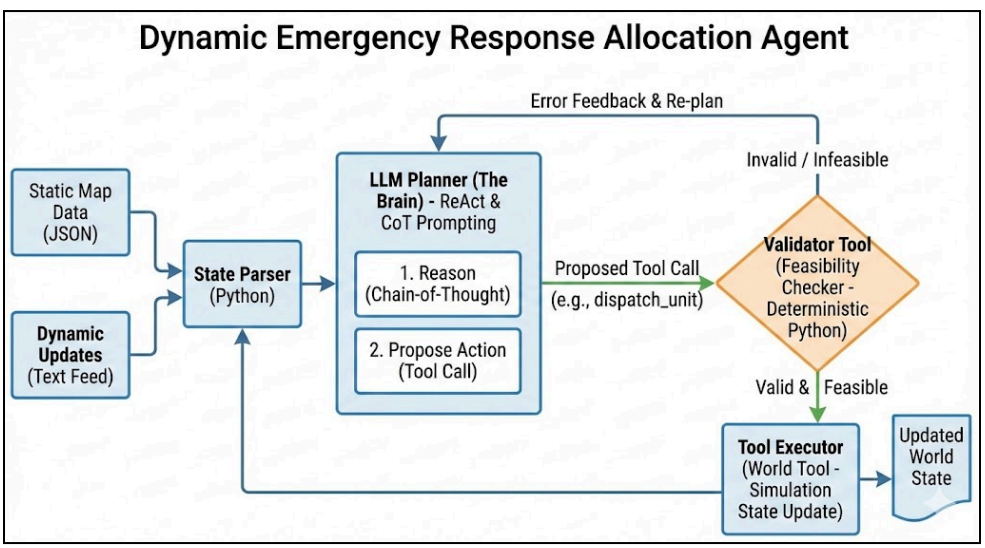
Subjective metrics are not applicable. In mission-critical OR, resource allocation is bound by deterministic mathematics. Using manual human grading destroys benchmark scalability and introduces uncontrolled bias.

4. Benchmark Independence

RescueBench functions as a standardized dataset of 20 static JSON scenario files paired with an open-source script. By publishing a strict JSON schema definition, any researcher can plug their agent's output array into our script to receive VRPTW performance scores.

Part III: Agent Design

1. Agent Architecture



The agent utilizes a custom, native Python orchestration loop following a constrained ReAct (Reasoning and Acting) paradigm. Key components include:

1. <u>The Planner</u>: The LLM core reasoning engine that calculates priority weights and decides on a routing strategy.

2. The Tool Executor: A standard interface using the Model Context Protocol (MCP) allowing the agent to interact with the simulation (e.g., dispatching vehicles).
3. The Memory: A system tracking conversational history and previous tool calls so the agent does not repeat mistakes.
4. The Validator: A deterministic safety layer checking the agent's spatial math and logic before any action is finalized in the simulation.

2. Technical Approach

The agent processes static city topologies and live crisis feeds, outputting structured JSON tool calls. We employ Chain-of-Thought (CoT) prompting, requiring the LLM to explicitly explain its mathematical reasoning step-by-step before it acts. If the agent hallucinates a physically impossible move, the Validator catches the error and sends a clear message back into the context window (e.g., “Error: That road is flooded”), triggering a Self-Correction Loop where the agent calculates a detour instead of hallucinating a success.

3. Baseline Comparisons

The agent is evaluated against three baselines using the exact same 20 scenarios and scoring scripts:

- ➔ Greedy Heuristic Solver: Assigns nearest available units to highest-severity incidents, proving if LLM strategic foresight adds value over basic math.
- ➔ Standard Zero-Shot LLM: A standard model generating a one-pass plan without our tool validator or feedback loops, proving the absolute necessity of error-correction.
- ➔ Ablated Agent: Our architecture, but stripped of its ability to process dynamic mid-mission updates.

Part IV: Resources and Timeline

Resources

We will utilize the Claude API for LLM reasoning, evaluating Claude Opus 4.6 for complex planning, Sonnet 4.6 for standard runs, and Haiku 4.6 for rapid testing. Our architecture relies on a custom World Tool to interface with the JSON map and a Validator Tool to deterministically check physical constraints and logic. The simulated JSON world will be generated locally to allow for complex dynamic scenarios, with a stretch goal of mapping the Chicago Loop using open-source data. Total API costs are estimated at $81, assuming 2 million input and 5 million output tokens.

Timeline

March 5 (Update 1): Create base JSON worlds, write the first end-to-end scenario, and implement the World and Validator tools.

March 26 (Update 2): Complete the first 5 tiered benchmark problems, run a naive baseline attempt, and reevaluate prompting strategies.

Mid-April (Update 3): Expand to 15 benchmark problems, test dynamic update scenarios, and build the memory component.

May 5 (Update 4): Finalize all 20 tasks, evaluate the agent across all scenarios using objective Python scripts, compare Claude models, and complete final optimizations.

References

[1] Song, P., Han, Y., & Goodman, N. (2026). “Large Language Model Reasoning Failures.” arXiv preprint arXiv:2602.06176.

[2] Jiang, M., et al. (2025). “An Agentic Framework with LLMs for Solving Complex Vehicle Routing Problems.” arXiv preprint arXiv:2510.16701.

[3] Chen, X., et al. (2025). “Multi-Objective Infeasibility Diagnosis for Routing Problems Using Large Language Models.” arXiv preprint arXiv:2508.03406.

[4] Liu, Y., et al. (2025). “Large Language Models in Operations Research: Methods, Applications, and Challenges.” arXiv preprint arXiv:2509.18180.